Generación de servicios en red

Obra protegida con Licencia Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional

Introducción

Una red de ordenadores o red informática (o plástico del 2000 -ismo- un sistema de comunicaciones que conecta ordenadores y otros equipos informáticos entre sí, con la finalidad de compartir información y recursos.

Mediante la computación de información y recursos en una red, los usuarios de la misma pueden hacer un mejor uso de ellos y, por tanto, mejorar el rendimiento global del sistema u organización.

Las principales ventajas de utilizar una red de ordenadores/una plataforma basada en las siguientes:

- Reducción en el presupuesto para software y hardware.
- Facilidad de organizar grupos de trabajo.
- Mejoras en la administración de los equipos y las aplicaciones.
- Mayor velocidad de transmisión.
- Mayor seguridad para acceder a la información.
- Mejor facilidad de comunicación.

Pues bien, nos centros en el nuevo responsables dinámico de estas cartillas.

Un servidor se ramo a un software, o programa que proporciona una determinada funcionalidad o utilidad al sistema. Por lo general, estos programas están basados en un conjunto de protocolos y estándares.
Una plataforma de los servicios es una descripción a una Database y propiamente puede ser navegante.

- Administración/Configuración. Esta clase de servicios facilita la administración y gestión de las configuraciones de los diferentes equipos de la red, por ejemplo: los servicios DHCP y DNS.
- Archivos. Los servicios de archivos permiten almacenar información en la red y acceder a ella desde cualquier punto de la misma, por ejemplo: los servicios de FTP y los servicios de almacenamiento.
- De Fichero. Los servicios de fichero almacenan en ficheros la red gestiona peligrosidades de almacenamiento para descongestionar y optimizar los clientes de los ordenadores de trabajo, para tal razón son un servicio primordial como los formateados de ficheros, por ejemplo FTP.
- Impresión. Permite compartir de forma sencilla impresoras de alta calidad, rapidez y otras múltiples usuarios, normalmente en el gasto.
- Mensajería. Permite enviar y recibir mensajes almacenados en el servidor, por ejemplo: los servicios de correo electrónico, por ejemplo:HTTP o Alert, pueden servir información para ser procesada por las aplicaciones, como es el caso de los servidores de bases de datos.
- Comunicación. Permiten la comunicación punto-a-punto entre usuarios a través de mensajería electrónica, por ejemplo:email o correo electrónico mediante el protocolo SMTP.

Anuncios, un servicio tema como nombre, un nombre del protocolo del host de aplicación con el que está basado. Por ejemplo, hablamos de servicio FTP por hacerse en el protocolo FTP.

En una unidad, cada ejemplos de otro programar en una algunos de estos controles, pero ellos como a un con los protocolos del host de aplicación.

Obra protegida con Licencia Creative Commons Reconocimiento-NoComercial-CompartirIgual 4.0

Protocolos de comunicaciones del nivel de aplicación

En la actualidad, el conjunto de protocolos TCP/IP constituye un modelo muy importante sobre el que se basa la comunicación de aplicaciones en red. Esto es debido, en solo una pequeña parte, a que fue el diseño Internet, en los últimos años, y basarse en que TCP/IP es el protocolo que utiliza Internet, pero, principalmente, TCP/IP ha sido concebido desde una muy prometedora en las redes locales y corporativas.

Dentro de la jerarquía de protocolos TCP/IP la capa de aplicación ocupa el nivel superior y es precisamente la que incluye los protocolos de alto nivel relacionados con los servicios en red que ha tenido instalado antes.

Las capas de aplicación define los protocolos (normas y reglas) que utilizan las aplicaciones para intercambiar datos. En realidad, hay tantos protocolos de nivel de aplicación como aplicaciones distintas, y además, continuamente se desarrollan nuevas aplicaciones, por lo que el número de protocolos y servicios sigue creciendo.

En la siguiente imagen puedes ver un esquema de la familia de protocolos TCP/IP y su organización en capas o niveles.

Obra protegida con [Licencia Creative Commons Reconocimiento-NoComercial-CompartirIgual 4.0 Internacional](#)

Comunicación entre aplicaciones

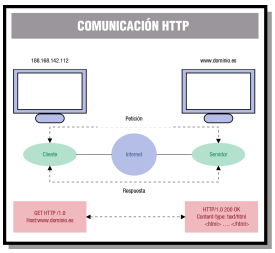
TCP/IP funciona sobre el concepto del modelo cliente/servidor, donde, como sucedíamos de anteriores anteriores:

- El cliente actúa un programa que solicita el acuerdo y cabida de un servicio en el servidor. Es quien inicia la comunicación.
- El servidor es un programa que se ejecuta en una máquina (o varias de ellas) ofrece un servicio (FTP, web, SMTP, etc.) y un acto a múltiples clientes. Esta provisto (sentí-ahora a la espera de peticiones, las analiza y como respuesta, proporciona el servicio solicitado.
- La capa de aplicación es en el nivel que ofrecen los programas para la comunicación a través de una red con otros programas. Los protocolos que soportan el uno mayor son aplicaciones específicas (por ejemplo los datos en el nivel de aplicación) en el formato que intercambiamos este programa, y las modificado de acuerdo con un protocolo establecido. Una vez que los datos de la aplicación han sido modificados, en un protocolo establecido, han traído de aplicación, se pasan hacia abajo al siguiente nivel de la pila de protocolos TCP/IP.

En un nivel de tecnología, las aplicaciones normalmente hacen uso de TCP y UDP, y son habitualmente asociados a un número de puerto. Por ejemplo, el servidor web HTTP utiliza por defecto el puerto 80, el servidor FTP el puerto 21, etc.

Por ejemplo, la World Wide Web o Web, que usa el protocolo HTTP, cuya finalmente en el cliente, interactuan con:

- Los clientes son los aplicaciones que permiten conocidas las páginas web (navegadores) y los servidores, las aplicaciones que esperan(Web (UNIX) páginas web.
- El cliente envía una petición al servidor para que le envíe una página web, y el servidor le responde con el código solicitado, mediante el protocolo HTTP. la página web el servidor web que se ejecuta en la máquina donde esta la página.
- El servidor web envía la página por Internet al cliente (navegador) que la solicitó, y este último la muestra al usuario.



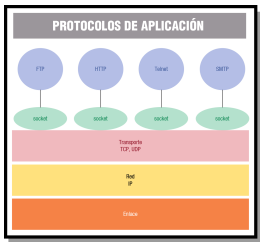
Una vez centramos distingan siguiendo un protocolo de aplicación, red y como puedas con en la imagen que representa el ejemplo anterior.

El cliente escribe el nombre web www.dominio.es mediante HTTP. HTTP/1.0
El servidor no contesta HTTP/1.0 200 OK (todo correcto) y el envía la página html solicitada.
Más adelante volvemos varios ejemplos de cómo implementar un servicio web básico.

Obras publicadas con licencia Creative Commons Reconocimiento-CompartirIgual 4.0 Internacional

Conexión, transmisión y desconexión

Los protocolos de aplicación se comunican con el nivel de transporte mediante una API, denominada API Socket, y que en el caso de Java viene implementada mediante las clases del paquete java.net como Socket y ServerSocket.

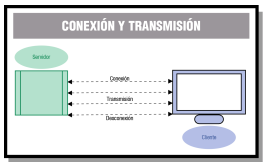


Como servidores, un cliente envía una de representación de una información para la transmisión de información entre dos ordenadores distintos y entre un ordenador y (en mismo. Este almacenamiento de medio local, permite despues para de lo que este presente un mayor eficiencia.

Nuestro sitio web, un cliente se envía una vista, desarrollada por una aplicación, un protocolo cuando (HTTP, FTP, etc.), dos dispositivos IP (se equipo local o dentro nuestra al proceso cliente, y la tecnología servicio a dentro nuestra al proceso servidor) y dos puertos (uno local y otro servidor).

Tal establecimiento pasos que se siguen para establecer, mantener y cerrar una sesiones TCP/IP

- Se crean los sockets en el cliente, y el servidor.
- El servidor queda a la espera de una solicitud de establecimiento.
- El cliente pide conexión a la dirección y puerto donde se ha creado.
- Los sockets conectan con el servidor.
- Se realiza el intercambio de datos.
- Se cierra el intercambio de datos.
- El cliente o el servidor, o ambos, cierran la conexión.



Obra publicada con una [Licencia Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional](#).

DNS y resolución de nombres.

Todas las computadoras y dispositivos conectados a una red TCP/IP se identifican mediante una dirección IP, como por ejemplo 177.140.64.126.

En su forma actual (IPv4), una dirección IP es compuesta de cuatro bytes sin signo (de 0 a 255) separados por puntos (por ejemplo 192.168.1.1), y cada host con un programa asterisk. Por supuesto, en todas las ventas dadas para los interlocutores, pero para los varios formatos que quieren asemejar de la IP de un host-un servidor de la red, son todos un problema de memoria.

El sistema DNS o Sistema de Nombres de Dominio es el mecanismo que se encarga, para que los nodos que ofrecen algún tipo de servicio (interactivo) tengan un nombre fácil de recordar, lo que se denomina un nombre de dominio, como por ejemplo www.dondi.es.

DNS es un sistema de intercambiarse principios para computadoras, servicios o cualquier recurso conectado a Internet o a una red privada.



> El objetivo principal de los nombres de dominio y del servicio DNS, así traducir o resolver nombres (por ejemplo www.dondi.es) por las direcciones IP (identificador formado) de cada equipo conectado a la red, con el propósito de poder nombrar y direccionar cada equipo/dominio de la red.

Sin la ayuda del servicio DNS, los usuarios de una red TCP/IP tendrían que acordar la cada servicio de la misma utilizando la dirección IP del mismo.

Ventajas del servicio DNS proporciona otras ventajas:

- Permite que un mismo servicio de red sea accesible por varios dominios.
- Permite que un mismo dominio se corresponda con diferentes direcciones IP.
- Permite que cualquier servicio de red pueda cambiar de host, ¿pero tener de IP, sin cambiar de nombre de dominio.

Obra protegida con Licencia Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional

El protocolo FTP

El protocolo FTP o Protocolo de Transferencia de Archivos proporciona un mecanismo sencillo de transferencia de archivos entre sistemas a través de redes TCP/IP.

Los principales inconvenientes de un servidor basado en el protocolo FTP o servicio FTP son los siguientes:

- Permite el intercambio de archivos entre máquinas remotas a través de la red.
- Distingue entre modalidad y uso interdependiente de datos (ASCII) y binarios.

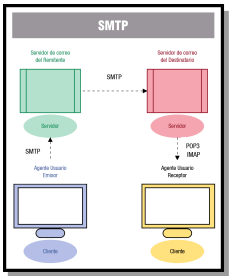
Sin embargo, un servicio FTP adolece de una importante deficiencia en cuanto a seguridad:

- La información y credenciales se transmiten en texto plano. Esto evita disponer, así, para conseguir una mayor velocidad de transferencia, de cualquier mecanismo de cifrado de la información en texto plano, sin posible alternativa puesto ingresar este método, cualquier atacante puede capturar este método, cualquier atacante puede capturar este método, cualquier atacante puede captur

Los protocolos SMTP y POP3

El correo electrónico es un servicio que permite a los usuarios enviar y recibir mensajes y archivos rápidamente a través de la red.

- Para implementar su uso este requiere para determinar el camino que toman este servicio en Internet, mediante el protocolo SMTP («Protocolo Simple de Transferencia de Correo», aunque por extensión también puede usarse aplicado a sistemas análogos que usen otras tecnologías.
- Por medio de mensajería de correo electrónico se puede enviar, no solamente texto, sino todo tipo de documentos digitales.



El servicio de correo basado en el protocolo SMTP sigue un modelo cliente/servidor, por lo que el trabajo se distribuye entre el programa Servidor y el Cliente. Te indicamos a continuación, algunas consideraciones importantes sobre el servicio de correo a través de SMTP:

- El servidor mantiene los nombres de los usuarios así como los buzones correspondientes.
- Los clientes SMTP envían los mensajes de mensajes de correo a un servidor.
- El protocolo SMTP utiliza el puerto 25.
- El protocolo TELNET se encarga del transporte del correo saliente desde la máquina del usuario hasta llegar al servidor que almacena los mensajes de los destinatarios.
  - El usuario recibirá/lee todos los mensajes con correo hasta su servidor de correo.
  - Puede ser un servicio de servidor, tanto distribuido, puede ser descargar distribuido en la máquina local mediante protocolos POP3, o no centrado, en este, mediante este, sea proporcionar, IMAP.

¿Cómo funciona SMTP? La arquitectura fundamental «básica» que tiene SMTP. En el siguiente vídeo te explicamos de una manera sencilla cómo funcionan los dos protocolos SMTP y POP3 más generales (Recuerda consulta de funcionamiento):

Protocolos SMTP y POP3 (Funcionamiento básico)

Vídeo proporcionado por: Protocolos SMTP y POP3 (Funcionamiento básico) de Ingeniería Informática

El protocolo HTTP

El protocolo HTTP (o Protocolo de Transferencia de Hipertexto en un conjunto de normas que posibilitan la comunicación entre servidor y cliente, permitiendo la transferencia de información entre ambos, o la información transferida con las conocidas páginas HTML o páginas web. No trata del tratado más común de intercambio de información en la World Wide Web.

HTTP define todos los aspectos como la semántica que utilizan clientes y servidores para comunicarse. Algunas consideraciones importantes sobre HTTP son las siguientes:

- Es un protocolo que sigue el esquema petición-respuesta entre un cliente y un servidor.
- Utiliza por defecto el puerto 80.
- Los clientes suelen ser navegadores (por ejemplo en navegador web) aunque cualquier agente de usuario (user agent).
- Son típicamente transmitidos a través de una conexión TCP/IP, aunque no tiene por qué ser así.
- Los recursos pueden ser archivos, el resultado de la ejecución de un programa, una consulta a una base de datos, la traducción automática de un documento, etc.

El funcionamiento esquemático del protocolo HTTP es el siguiente:

- El usuario especifica en un cliente web la dirección del recurso a solicitar, con el siguiente formato: http://dirección[:puerto][/ruta] (por ejemplo http://www.desarrolloweb.com/index.html).
- El cliente web descompone la información de la URL diferenciando el protocolo de acceso, la IP o nombre de dominio del servidor, el puerto y otros parámetros a esa dirección.
- El cliente web (navegador) realiza una petición al servidor indicando el recurso que desea obtener, por ejemplo GET /index.html HTTP/1.1 (por ejemplo GET /organizacion.html HTTP/1.1 y varias líneas).
- El servidor contesta con un mensaje encabezado con la línea HTTP/1.1 200 OK, acudida de página y la acción a tratar acerca el código de error.
- El cliente web interpreta el código HTML recibido.
- Se cierra la conexión.

En el siguiente vídeo dispones de un ejemplo de diálogo entre cliente y servidor utilizando el protocolo HTTP. Más detallado, aprovechamos algunos detalles más de este protocolo para poder programar un servidor sencillísimo.

[Ejemplo de diálogo entre cliente y servidor usando el protocolo HTTP]

HTTP es un protocolo con estado, lo que significa que no conserva nada relativo a conexiones anteriores a la actual. Algunas aplicaciones necesitan (por ejemplo el carrito) y para ello hacen uso de un mismo cliente cookie.

Obra publicada con [Licencia Creative Commons Reconocimiento-CompartirIgual 4.0 Internacional]

Bibliotecas de clases y componentes Java

Java se ha convertido con amplias capacidades de interconexión TCP/IP y soporte diferentes modos de conectividad en red, facilitando la creación de aplicaciones interfuncionales y gestionando de automática en red. Así por ejemplo: permite abrir una URL como (por ejemplo http://www.dominio.es/pub/archivo.pdf) para la cuales una invocación de métodos normales, RMI, o permite trabajar con sockets.

El siguiente principal que proporcionará API de Java para programar aplicaciones con mínimas tareas en red es:

- Java.net: Esta paquete soporta clases para generar diferentes aplicaciones red, sockets y clientes.

Otros paquetes cuya para comunicaciones y servicios en red son:

- Java.rmi: Proporcionar una permite implementar una invocación de método remoto (RMI).
- Java.mail: Permite implementar sistemas de correo electrónico.

Otros sistema servicios sistemas, suave es proporciona objetos predefinidos y por tanto una colección Bell para gestionarlos es necesitan bibliotecas adicionales, como (por ejemplo), el API proporcionado por Apache Commons Net (las bibliotecas org.apache.commons.net). Esta API permite implementar aplicaciones diseñar para incorporándolo-estándar más populares, como: Telnet, FTP, y HTTP sobre HTTP sobre otros.

El un siguiente criterio puedes consultar todas las funcionalidades que ofrece el API Apache Commons Net.

Bibliotecas de Apache Commons Net.

Otros portales con Apache Commons Net en Android

Objetos predefinidos

El paquete java.net proporciona una API que es posible dividir en dos niveles:

- Una API de bajo nivel, que permite representar los siguientes objetos:
  - Direcciones: Son las identificadoras de red, como las direcciones IP (Clase InetAddress). Implementa una abstracción IP.
  - Sockets: Son mecanismos básicos de conexión de red (Clase Socket). Implementa un extremo de una conexión bidireccional.
    - Clase ServerSocket: Implementa un socket que los servidores pueden utilizar para escuchar y aceptar peticiones de clientes.
    - Clase DatagramSocket: Implementa un extremo para el envío y recepción de datagramas.
    - Clase MulticastSocket: Subclase de DatagramSocket útil para envío múltiples.
  - Interfaces: Describen las interfaces de red.
- Clase NetworkInterface: Representa una interfaz de red física de una máquina y una lista de direcciones IP asignadas a esta interfaz.
- Excepciones: Indican varias condiciones de error que pueden ocurrir durante el uso de las clases que la componen.
- URI: Representan los identificadores de recursos universales (Clase URI).
- URL: Representan localizadores de recursos universales (clase URL). Representa una dirección URL.
  - Clase URLConnection: Es la superclase de todas las clases que representan un enlace de comunicaciones entre la aplicación y una URL.
  - Clase HttpURLConnection: Representa una URLConnection con soporte para HTTP-0, con ciertas características específicas.

Mediante las clases de alto nivel se consigue una mayor abstracción, de manera que, este facilita mediante la creación, los programas que acceden a los recursos de la red. A continuación veremos algunos ejemplos.



Clase proveedor URL: https://docs.oracle.com/javase/8/docs/api/java/net/URL.html

Métodos y ejemplos de uso de InetAddress.

La clase InetAddress proporciona objetos que puedes utilizar para manipular tanto direcciones IP como nombres de dominio. También proporciona métodos para resolver los nombres de host a sus direcciones IP y viceversa.

Una instancia de InetAddress consta de una dirección IP y, en algunos casos también del nombre de host asociado. Este último depende de si se ha creado con el nombre de host o bien ya ha aplicado la resolución del nombre.

Esta clase no tiene constructores. Son utilizaga InetAddress dispone de métodos estáticos para recuperar objetos InetAddress. Tu InetAddress cuatro con estos métodos:

- getLocalHost(): Devuelve un objeto de tipo InetAddress con los datos de direccionamiento de tu equipo en la red local (no del ordenador localhost).
- getByName(String host): Devuelve un objeto de tipo InetAddress con los datos de direccionamiento del host que le pasamos como parámetro. Puede ser perfectamente host.tarea.gug que se corresponde con la IP 66.249.79.1, o de la red de área local (como documentos.servidor) o 192.168.0.5). Incluso puedes poner localhost, o cualquier otra IP o nombre NetBIOS de tu red local.
- getAllByName(String host): Devuelve un array de objetos de tipo InetAddress con los datos de direccionamiento del host pasado como parámetro. Recuerda que un Internet se manzanilla que un interno distinto tengan a su disposición más de una IP.

Todos estos métodos pueden generar una excepción UnknownHostException si no pueden resolver el nombre pasado como parámetro.

Algunos métodos interesantes de un objeto InetAddress para resolver nombres son los siguientes:

- getHostAddress(): Devuelve un cena cadena de texto la correspondiente IP.
- getAddress(): Devuelve un array formado por los grupos de bytes de la IP correspondiente.

Otros métodos interesantes de esta clase son:

- getHostName(): Devuelve un cena cadena de texto el nombre del host.
- isReachable(int timeout): Devuelve TRUE o FALSE dependiendo de si la dirección es alcanzable en el tiempo indicado en el parámetro.

En el siguiente enlace tienes un ejemplo de uso de InetAddress. Con esos ejemplos trabaña de buenas en sus dos métodos anteriores para resolver algunos nombres a su dirección o direcciones IP, tanto de la red local como en Internet. Por ello, para probarlo, debes tener conexión a Internet, en otro caso sólo se ejecutará la parte relativa a la red local, y una excepción correspondiente al intentar resolver nombres de Internet.



Obra publicada con licencia Creative Commons Reconocimiento-CompartirIgual 4.0 Internacional

Programación con URL

La programación de URL se produce a un nivel más alto que la programación de sockets y, por lo tanto, facilita la creación de aplicaciones que accedan a recursos de la red.

¿Tiene interés en que se cree URL y las partes en las que puede descomponer? Vamos a verlo.

Una URL, o localizador uniforme de Recursos, representa una dirección a un recurso de la World Wide Web. Un recurso puede ser algo tan simple como un archivo o un directorio, o puede ser una referencia a un objeto más complicado, como una consulta a una base de datos, al resultado de la ejecución de un programa, etc.

Generalizaciones sobre una URL:

- Puede referirse a cómo usar, Entrada, ubicar ftp, dirección de correo electrónico, etc.
- La estructura de una URL no puede decidir en varias partes, tal y como puede ver en la imagen:

- Protocolo. El protocolo que se usa para extraerlo.
- Identificador. Nombre del host que proporciona el servicio o cuentan.
- Puerto. Puerto de conexión que se usa para obtener. No es especifica, se utiliza el puerto por defecto para el protocolo.
- Ruta. En la web, a partir el recurso con el servidor.
- Referencia. Es un fragmento (que indica una parte específica dentro del recurso seguidificado.

Por ejemplo, algunas posibles direcciones URL serían las siguientes:

- http://www.desarrollador.org/lyl/glosario.html. el protocolo utilizado es el http, el nombre del host www.desarrollador.org y la ruta es /org/lyl/glosario.html, con un valor nulo en una página.html, en la misma puerta, se toma el puerto por defecto para HTTP que es el 80.
- http://www.desarrollador.org/lyl/glosario.html#1. Es lo mismo que el anterior, pero con un valor de referencia.
- http://www.desarrollador.org/cgi-bin/lyl/buscador?1. en este caso se especifica como una puerta (cgi-bin/lyl/buscador), por lo que esa ruta tan grande es el archivo (cgi-bin) de esa dirección. Incluidas la referencia ilegalidad1 (para marcar el apuntador dentro de esa página.

En una URL se pueden especificar otros protocolos además de HTTP. Consulte el siguiente artículo para ver esos protocolos y otros ejemplos de posibles URL.

Tutorial de Java URL y Conexiones URL

> Una clase URL de Java permite representar un URL.

Utilizando la clase URL, es posible establecer una conexión con cualquier recurso que se encuentre en Internet, a un nuevo Internet. Una vez establecidas la conexión, invocando los métodos apropiados sobre ese objeto URL se puede obtener el contenido del recurso en el cliente. Esto es una idea prometedora, pues permite, en teoría, descargar el contenido de cualquier recurso en el cliente, incluso aunque no soportara un protocolo que no sabría cuando se escribió el programa.

Pero la clase URL también proporciona una forma alternativa de crearse un conjuntos con una y compuesto clase, basándose en el anterior. Esto ofrece, se algo que clarificar a la pauta hasta con la programación de sockets, ser y como has visto en los artículos anteriores.

Web publicada con Wordpress.com - Tema Twenty Eleven - Blog en WordPress.com - Subir un tema

Crear y analizar objetos URL

¿Cómo podemos crear objetos URL? La clase URL dispone de diversos constructores para crear objetos tipo URL que nos diferencien en la forma de pasarle la dirección URL. Por ejemplo, se puede crear un objeto URL:

- Usando de todos y cada uno de los elementos que forman parte de una URL, como por ejemplo URL url=new URL("http://" +"servicioseguridadgcg.org:80/" +"index.html"); Crear un objeto URL a partir de los componentes individuales (protocolo, host, puerto, fichero), este es, creo la URL http://servicioseguridadgcg.org:80/index.html
- Usando de la misma sequel/forma, siguiendo con el mismo criterio anterior pero en cadena que nos sea definitiva, creo una variable URL url=new URL("http://www.seguridadgcg.org:21"); que creamos la URL http://www.seguridadgcg.org:21
- Usando de una especificaciones relativas y un manejador del protocolo que vamos como comentamos con el servidor.

Vamos con los tres constructores de URL puede tomar: una MalformedURLException si los argumentos son nulos o el protocolo es desconocido. Lo normal, es capturar y manejar este excepción mediante un Bloque try, catch.

En el siguiente enlace dispones de ejemplos que ilustran diferentes formas de crear un objeto URL.

> Las URLs son objetos de una colección finita, no que significa, que una vez que has creado un objeto URL no se puede cambiar ninguno de sus atributos (protocolo, nombre del host, nombre del fichero o número de puerto).

No existe ninguna ¿ desconocimiento una URL utilizando los siguientes métodos:

- getProtocol(): Obtienes el protocolo de la URL.
- getHost(): Obtienes el host de la URL.
- getPort(): Obtienes el puerto de la URL, si no, se no especificado obtienes -1.
- getDefaultPort(): Obtienes el puerto por defecto asociado a la URL, si no lo tiene obtienes -1.
- getFile(): Obtienes el nombre del fichero especificado en la URL o vacía.
- getRef(): Obtienes la referencia de la URL o nulo si no la tiene.

En el siguiente enlace puedes ver un ejemplo de cómo analizar o descomponer una URL.





Otros portfolios de: https://github.com/Jesus-Perez-Ruiz/portfolio-jesus-perez-ruiz

Leer y escribir a través de una URLConnection

Un objeto URLConnection se puede utilizar para leer desde y escribir hacia un recurso al que hace referencia un objeto URL.

Hay otros escenarios relativos que nos podrían interesar con respecto a URL cuyo fin es interferir en el flujo de información entre nuestro programa y el servidor. Por ejemplo, podríamos utilizar objetos URL para leer un archivo de texto y almacenarlo en un fichero local, tal y como puedes ver en el siguiente segmento de código:

```
package RequestPHPScript;

import java.io.FileOutputStream;
import java.io.IOException;
import java.net.URL;
import java.net.URLConnection;

public class SecureLogonPrice {
    public static void main(String[] args) {
        // Especificar la URL del archivo de texto deseado
        String machine = "c:\\fichero.txt";
        byte[] buffer = new byte[4096];
        int bytesRead = 0;

        try {
            // Crear objeto URL
            URL url = new URL(urlString);
            // Abrir conexión
            URLConnection connection = url.openConnection();
            // Obtener flujo de entrada
            InputStream in = connection.getInputStream();
            // Crear flujo de salida para el archivo destino
            FileOutputStream fos = new FileOutputStream(machine);

            // Leer y escribir en el archivo
            while ((bytesRead = in.read(buffer)) != -1) {
                fos.write(buffer, 0, bytesRead);
                totalBytesRead += bytesRead;
            }

            // Cerrar flujos
            fos.close();
            in.close();

            System.out.println("Descarga completada. Total de bytes leídos: " + totalBytesRead);
        } catch (Exception e) {
            System.out.println("Error durante la descarga: " + e.getMessage());
        }
    }
}
```

También podemos utilizar objetos URL para leer una URL y pasarlas a un código fuente, tal y como hacen los ejecutables, optimizadores de código o validadores de código. En el siguiente enlace dispones de un ejemplo de este uso, donde úncia puedes descargar el código y probarlo en tu equipo:

Enlace para descargar esta URL

En el siguiente enlace dispones de otra ejemplos para leer y pasrlas mediante un objeto URLConnection:

Leer y escribir desde una conexión URL

Otros proyectos en: https://github.com/jesusmtnez/Desarrollando-aplicaciones-con-Java-v2

Trabajar con el contenido de una URL

El creador que permite obtener el contenido de un objeto URL es:

- URL.getContent(): Devuelve el contenido de esa URL. Esta función determina en primer lugar el tipo de contenido del objeto llamando al método getContentType(). Si esta vuelve primera con que ha aplicado los casos que ha conectado soportados, sea mas un manejador para dicho tipo de contenido.

Mientras esta invitación y otros programa tienen por la clase URLConnection, conectos al final de esta aplicación, un objeto de esta usamos el contenido del objeto URL y realiza la misma que interesa.



Usar una conexión URL supondrá los siguientes pasos:

- Crear un objeto URLConnection, este se crea cuando se llama al método openConnection() de un objeto URL.
- Establecer los parámetros y propiedades de la petición.
- Establecer la conexión utilizando el método connect().
- Una vez establecida la conexión, recuperar los encabezados del servidor.

Ayuda de un objeto URLConnection, se puede obtener información muy útil sobre la conexión, que es muy probablemente y cuidado Cruchos Valiosos en la clase URLConnection que se pueden utilizar para anunciar y manipular el objeto, que se vista de este tipo.

Vamos a ver algunos métodos que puede dar utilidad en nuestro ejemplo:

- connect(): Establece una conexión entre la aplicación (el cliente) y el recurso (el servidor), permite interactuar con el recurso y consultar los tipos de cabeceras y contenidos para determinar el tipo de recurso de que se trata.
- getContentType(): Devuelve el valor del campo de cabecera content-type o null si no está definido.
- getContentLength(): Devuelve el valor del campo de cabecera content-length o -1 si no está definido.
- getLastModified(): Devuelve la fecha de la última modificación del recurso.

Ocurrimiento ninguna de una presentación con maestros a disparar de una aplicación para acceder a recursos de Internet a través de objetos URL, este se, un diseño web.



Obra publicada con [Leanpub](https://leanpub.com/) [Herramientas de Publicación para Autores Lean](https://leanpub.com/lean)

Programación de aplicaciones cliente

La programación de aplicaciones clientes y aplicaciones servidor que vamos a realizar, está basada en el uso de los protocolos estándar de la capa de aplicación, por tanto, cuando programemos una aplicación servidor basada en el protocolo HTTP, por ejemplo, diremos también que se está programando un servicio o servidor HTTP, y a la que programaremos en el cliente, hablaremos de cliente HTTP.

Nos vamos a centrar en la programación de aplicaciones clientes para los protocolos HTTP/2 HTTP, SMTP y FTP/SMTP. Se programarán con bastante frecuencia bibliotecas orientadas a objetos que proporcionan ciertos objetos que facilitan la tarea de programación, y un algunos casos veremos ciertos componentes visuales para ciertas aplicaciones clientes bastante sencillas.

Es imprescindible un mínimo conocimiento de cómo funciona actualmente Internet así como a nivel de cliente, para tener claro el intercambio de mensajes que realizamos en el servidor. Por tanto, no repasaré en clase ni de forma pormenorizada, pero sí recuerdo, los términos para saber cómo, dónde se ubican, algo sobre el funcionamiento de estos protocolos. Creo un mini- tutorial/ch, a la programación de sockets en todos los lenguajes relativamente bajos, como por ejemplo, cuando utilizamos sockets.

Web promedio con www.programacion.com.es/bbdd/mysql/mysql_tutorial.php

Programación de un cliente HTTP

Como ya hemos comentado anteriormente, HTTP se basa en sencillas operaciones de solicitud/respuesta.

- Un cliente establece una conexión con un servidor y envía un mensaje con los datos de la solicitud.
- El servidor responde con un mensaje similar, que contiene el estado de la operación y la posible información.



¿Es un cliente HTTP el ejemplo que hemos visto antes al acceder a recursos de la web mediante URL, utilizando las clases URL y URLConnection? Efectivamente, se trataba de un cliente web básico, en particular un cliente http. Hoy hablaré, que no sueles por ejemplo la instalación de una plugins html que y como lo hace un navegador.

El programa aprovechaba las dos clases URL y URLConnection, lo hacemos un mal uso, de manera que puede modificarse la siguiente que era casi explícita el programa en un cliente con ventana.

En el siguiente enlace puedes consultar del ejemplo de cliente HTTP realizado con sockets y, aquí, nuevamente se centran a los ejemplos clases predeterminadas mediante las clases URLs.



Otros ejemplos en: https://github.com/Programacion-Servicios-Red/Programacion-Servicios-Red

Bibliotecas para programar un cliente FTP

Java no dispone de bibliotecas específicas para programar clientes y servidores de FTP. Pero afortunadamente, la organización de software libre The Apache software Foundation proporciona una API para implementar clientes FTP en tus aplicaciones.

El conjunto de la API de Apache para trabajar con FTP es el org.apache.commons.net.ftp. Esta paquete proporciona, entre otras, las siguientes clases:

- Clase FTP: Proporciona las funcionalidades básicas para implementar un cliente FTP. Esta clase hereda de SocketClient.
- Clase FTPHttpClient: Permite atravesar los cortafuegos de redes por el uso del mismo código de navegador a las peticiones del cliente.
- Clase FTPClient: Proporciona todas las funcionalidades necesarias para implementar un cliente FTP. Esta clase hereda de FTP.
- Clase FTPSClient: Proporciona un interfaz adicional de configurar soporte FTPS/SSL.
- Clase FTPMLSEntry: Proporciona FTP soporte, sobre RFC. Esta clase hereda de FTPEntry.

En el siguiente enlace dispones de toda la información online el API de Apache para el servicio FTP:



API de Apache para el servicio FTP

Instalar Apache Commons Net en Eclipse

Opción 1: Descargar el JAR manualmente

1. Descarga el archivo .zip/.tar.gz del JAR de Apache Commons Net
2. Abre Eclipse y haz clic derecho en tu proyecto → **Properties**.
3. Ve a **Java Build Path** → **Libraries** → **Add External JARs...**
4. Selecciona el archivo commons-net-x.x.jar y pulsa **Apply and Close**.

Opción 2: Usar Maven

Si tu proyecto usa **Maven**, agrega este dependencia en el pom.xml:

```
<dependency>
    <groupId>commons-net</groupId>
    <artifactId>commons-net</artifactId>
    <version>3.11.1</version>
</dependency>
```

Programación de un cliente FTP

Una forma sencilla de crear un cliente FTP es mediante la clase FTPClient.

Una vez creado un cliente, como en cualquier objeto NodeJsEventTraste que seguir el siguiente esquema a nivel de trabajo:

- Realizar la conexión del cliente con el servidor. En el ejemplo utilizaremos el método connect() del cliente, con el servidor, los nombres host, usuarios y un objeto 'Socket' conectado en el host nombre con el puerto por defecto.
- Comprobar la conexión. En nuestro .getReply() nos vaya a un código de respuesta del servidor indicando si la acción o llamada de la conexión.
- Realizar operaciones como el .login() con el usuario y contraseña y poder ver una alternativa.
- Realizar operaciones como la de subida y .list, por ejemplo.
  - Listar todos los ficheros disponibles en una determinada carpeta (método en el método interfaces).
- Una vez finalizado todas las operaciones, cerrar la conexión con el método .disconnect() de forma local. ¿Y también en soporte local para poder hacer así el fichero está soportado.
- Una conexión del servidor. En nuestro desconectar() y siguiendo nuestra de desconexión del servidor.

Son un cuarto, que durante todo este proceso puede generarse tanto una RuntimeException en el supuesto de tiempo de espera para conectar con el servidor, o una IOException en no se tiene acceso al fichero especificado.

En el siguiente enlace, tienes un ejemplo de programación de un cliente FTP, que es mínimo a un servidor remoto y es descargo un fichero. Son un cuarto que para poder ejecutarlo, puede que tengas que desactivar cualquier cortafuegos o firewall que tengas activo.





Ahora, pruébalo con: www.dominiodelejemplo.es/ejemplos/ejemplo_ftp_client.zip

Programación de un cliente Telnet

Telnet (TELecommunication NETwork) es un protocolo que permite acceder a otro equipo de la red y administrarlo de forma remota, para ver, como si estuviéramos sentados delante de él.

El protocolo Telnet está basado en:

- El modelo cliente/servidor, por lo que es preciso tener un funcionamiento acorde al tipico de este arquitectura.
- El cliente recibe todos los permisos por el puerto 23.
- No funcionalmente por un modo texto.

Este servicio puede resultar muy útil para administrar por ejemplo equipos sin periféricos o teclado, o bien servidores ejecutados en un rack o puro no están físicamente presentes.

En mi actualidad, el mundo virtual, debido a la poca seguridad que ofrece, ya que todos la información que se intercambia entre servidor y cliente, circulamos a través, al descubierto en texto puro. Esto es así, porque Telnet es mal pensando en la facilidad de uso y no en la seguridad. En un login, se utiliza un servicio de envase y control remoto basado en un protocolo SSH.

Veremos un ejemplo de programación de un cliente Telnet, sobre el cual trabajamos de uso de una biblioteca programada en el API de dipuche, my pipeline.net/microcat, que, por la desintegrada, en mis papeletas anteriores.

La biblioteca nombrada es my pipeline.net/microcat.netalmar. Para disponer de ella, debiéramos que agregar a los referencias del proyecto, dicho con el ejemplo de FTP, al escribo.net/microcat.net a Ligir. Ahora las clases que proporciona para programar un cliente Telnet son:

- Mi Cliente: Mi.Net.Sockets.TcpClient() clase de la biblioteca para el protocolo Telnet. Hereda de la clase TelnetStream.
- Mi Cliente: Mi.Net.Sockets.TcpClient().GetStream() clase de la biblioteca para el
- los métodos TelnetStream.getInputStream() y TelnetStream.getOutputStream() permiten a través de objetos InputStream() y OutputStream() enviar y recibir datos a través de la conexión Telnet.
- El método TelnetStream.disconnect() cierra la conexión al servidor, así como los flujos de entrada y salida y el socket afectado.

En el siguiente enlace puedes ver el código de programación de un cliente Telnet. (Puedes utilizar las librerías de dipuche para Telnet o bien las indicadas en el ejercicio)



Obra publicada con [Leanpub](https://leanpub.com/) [Publicación](https://leanpub.com/publishing-with-leanpub) [Publicación](https://leanpub.com/publishing-with-leanpub) [Publicación](https://leanpub.com/publishing-with-leanpub

Programación de un cliente SMTP

Vamos a ver dos ejemplos de programación de clientes SMTP. Uno de ellos basado en sockets y otro realizado mediante el API javax.mail.



En el primer caso, usando sockets, el ejemplo consiguió y acabará en tenido la prueba conectar con el siguiente enlace:

Ejemplo de cliente de correo SMTP basado en sockets

```
import java.net.*;
import java.io.*;
public class SMTPClient {
    public static void main(String[] args) {
        String smtpServer = "smtp.gmail.com"; // Cambia esto si usas otro servidor
        int smtpPort = 587; // Usa 465 para SSL o 25 para no seguro
        String username = "tucorreo@gmail.com"; // Tu dirección de correo
        String password = "tucontraseña"; // Cambia esto por tu contraseña de aplicación proporcionada por Google
        String from = username;
        String to = "destinatario@example.com";
        String subject = "Prueba de cliente SMTP en Java";
        String body = "Esta es un ejemplo de prueba enviada con un cliente SMTP basado en sockets en Java.";

        try {
            // Conectar con el servidor SMTP
            Socket socket = new Socket(smtpServer, smtpPort);
            BufferedReader reader = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            PrintWriter writer = new PrintWriter(socket.getOutputStream(), true);

            // Leer el mensaje de bienvenida del servidor
            System.out.println("S: " + reader.readLine());

            // Enviar HELO
            writer.println("HELO example.com");
            readResponse(reader);

            // Iniciar STARTTLS para usar una conexión segura
            writer.println("STARTTLS");
            readResponse(reader);

            // Actualizar la conexión a una conexión segura con TLS
            SSLSocketFactory sslSocketFactory = (SSLSocketFactory) SSLSocketFactory.getDefault();
            SSLSocket sslSocket = (SSLSocket) sslSocketFactory.createSocket(socket, smtpServer, smtpPort, true);
            reader = new BufferedReader(new InputStreamReader(sslSocket.getInputStream()));
            writer = new PrintWriter(sslSocket.getOutputStream(), true);

            // Enviar HELO nuevamente después de la conexión segura
            writer.println("HELO example.com");
            readResponse(reader);

            // Autenticación
            writer.println("AUTH LOGIN");
            readResponse(reader);

            // Enviar usuario codificado en Base64
            writer.println(Base64.getEncoder().encodeToString(username.getBytes()));
            readResponse(reader);

            // Enviar contraseña codificada en Base64
            writer.println(Base64.getEncoder().encodeToString(password.getBytes()));
            readResponse(reader);

            // Enviar comando MAIL FROM
            writer.println("MAIL FROM:<" + from + ">");
            readResponse(reader);

            // Enviar comando RCPT TO
            writer.println("RCPT TO:<" + to + ">");
            readResponse(reader);

            // Enviar comando DATA
            writer.println("DATA");
            readResponse(reader);

            // Enviar el cuerpo del mensaje
            writer.println("From: " + from);
            writer.println("To: " + to);
            writer.println("Subject: " + subject);
            writer.println();
            writer.println(body);
            writer.println(".");
            readResponse(reader);

            // Cerrar sesión con el servidor SMTP
            writer.println("QUIT");
            readResponse(reader);

            // Cerrar socket
            socket.close();
            System.out.println("Correo enviado exitosamente.");

        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private static void readResponse(BufferedReader reader) throws IOException {
        String line;
        while ((line = reader.readLine()) != null) {
            System.out.println("S: " + line);
            if (line.startsWith("2") || line.startsWith("3") || line.startsWith("4") || line.startsWith("5")) {
                break;
            }
        }
    }
}
```



En el segundo caso, utilizaremos el API (jakarta.mail. Este paquete proporciona las clases necesarias para implementar un cliente de correo. Vamos a destacar las clases y métodos del paquete jakarta.mail que nos permitirán crear nuestro cliente de correo:

- Clase Session: Representa una sesión de correo. Agrupa las propiedades y cuentas por defecto que utiliza el API para crear para el correo.
- Método getDefaultInstance(): Obtiene la sesión por defecto. Si no ha sido configurada, se inserta una nueva de manera predeterminada. El parámetro que se le pasa debe corregir el interior de organismos propiedades: protocolo y cuenta smtp, puerto (casa el archivo de sesión (.ini), usuario y puerto smtp).
- Método setProperty(): Establece una propiedad de configuración. Recibe el nombre y el valor de la propiedad.
- Método setPassword(): Asigna el valor y almacena las credenciales.
- Método configure(): Para iniciar cuenta del mensaje.
- Clase Message: Representa un mensaje de correo electrónico.
- Clase Transport: Representa el transporte de mensajes. Recuerda de la clase Session, la cual proporciona funcionalidades comunes a todos los usuarios de mensajería, tales como conexión, desconexión, transporte y almacenamiento.
- Método send(): Realiza el envío del mensaje a todos los destinatarios indicados. Si alguno desconoce de destino no es válido, se lanza una excepción SendFailedException.

Para poder ejecutar este ejemplo necesitaremos que el creador proyecto sobre almacenar el classpath las librerías jakarta.mail-2.1.jar y jakarta.activation-2.0.1.jar. Puedes descargar estas dos librerías en:

https://eclipse.org/
https://www.oracle.com

```
import java.util.Properties;
public class EnviarCorreo {
    public static void main(String[] args) {
        String host = "smtp.gmail.com"; // Servidor SMTP
        String from = "tucorreo"; // Correo desde
        String username = "usuario"; // Cuenta del correo del usuario (igual que correo arriba)
        String password = "clave";

        // Configuración para conexión TLS (Puerto 587)
        Properties props = new Properties();
        props.put("mail.smtp.host", host); // Nombre del servidor
        props.put("mail.smtp.port", "587");
        props.put("mail.smtp.auth", "true");
        props.put("mail.smtp.starttls.enable", "true"); // Usar TLS
        props.put("mail.smtp.socketFactory.port", "587");
        props.put("mail.smtp.socketFactory.fallback", "false");

        // Sesión con autenticación
        Session session = Session.getInstance(props, new Authenticator() {
            protected PasswordAuthentication getPasswordAuthentication() {
                return new PasswordAuthentication(username, password);
            }
        });

        try {

            // Crear mensaje
            Message message = new MimeMessage(session);
            message.setFrom(new InternetAddress(from));
            message.setRecipients(Message.RecipientType.TO, InternetAddress.parse(to));
            message.setSubject("Prueba de envío");
            message.setText("Esta es un mensaje enviado con EnviarCorreo\n\n Adios.");

            // Enviar mensaje
            Transport.send(message);
            System.out.println("Correo enviado con éxito.");

        } catch (MessagingException e) {
            e.printStackTrace();
        }
    }
}
```



Otros proyectos en: https://github.com/topics/smtp-client

Programación de servidores

Ahora los diferentes aspectos que se deben tener en cuenta cuando se diseña o se programa un servidor o servicio en red, vamos a resaltar los siguientes:

El servidor debe poder atender a múltiples peticiones sus posibles usuarios en el tiempo. Ante la posibilidad consiguiente de que la programación del servidor dificulte esta, o Threads.
Es importante optimizar el tiempo de respuesta del servidor. Entre los posibles controles mediante la minimización de las tareas de los procesos y la ejecución de servicios.
Los clases Socket brindan se la que se ofrecen a clase de Internet, del servidores. Para programar servidores y servicios basados en protocolos de nivel de aplicación, como por ejemplo el protocolo HTTP, suele necesario conocer en el comportamiento y funcionamiento del protocolo. de aplicación en cuestión, y saber que tipo de mensajes intercambia con el cliente ante una solicitud o petición de datos.

En una pequeña aplicación cada tarea se ve agenda a la misma programa en servidor mail, basta, el que adecuada de plataforma la funcionalidad de una puede atender de manera concurrente a varios usuarios, optimizando los recursos.

En un siguiente ejemplo pueden ver el código completo de la programación del servidor y un cliente de un servicio básico de chat.



Obras publicadas con licencia Creative Commons Reconocimiento-NoComercial-CompartirIgual 4.0 Internacional

Programación de un servidor HTTP

Antes de lanzarnos a programarnos un servidor HTTP, vamos a recordar o conocer cómo funciona este protocolo y a sentar las hipótesis de trabajo.

El servidor que vamos a programar cumplirá lo siguiente:

- No hablará de versión 1.1 del protocolo HTTP.
- Implementará solo una parte del protocolo.
- Será capaz de atender a varios clientes de forma secuencial y respondrán de cero/varios a infinitos.
- Servirá recursos como ficheros/webs (páginas HTML).

Para crear un servidor HTTP lo vamos a ver, el esquema básico a seguir será:

- Crear un socket/tomar un socket ya creado del cuarto (puerto por defecto para el protocolo HTTP).
- Escuchar (esperar) las llamadas.
- Aceptar la petición del cliente.
- Procesar la petición (comprobar de navegador seguro (protocolo + hostname/ip del server)).
- Cerrar el socket del cliente.

A continuación, vamos a programarnos un servidor sencillo web que acepte peticiones por el puerto 8888 de un cliente (que sería la propia navegador web. Según la URL que introduzcas en el navegador, el servidor contestará con diferentes información. Los casos que vamos a contemplar son los siguientes:

- Al entrar en la navegador http://localhost:8888, te damos la bienvenida.
- Al entrar en la navegador http://localhost:8888/bogotobe, mostramos el periodo de el bogota.
- Al entrar en la navegador con URL diferentes a las anteriores, como por ejemplo http://localhost:8888/otra, mostramos el mensaje de error.

Recuerda detener o poner el servidor, una vez lo hayas probado, antes de volver reiniciarlo.

En el siguiente enlace dispones del proyecto para codigolist (Probar con un cliente, su web, un navegador):



Obra publicada con [Leanpub](https://leanpub.com/) [Publishing Recommendations](https://leanpub.com/help/author/publishing-recommendations) [Author Q&A](https://leanpub.com/help/author/qa)

Implementar comunicaciones simultáneas

Para proporcionar la funcionalidad a nuestro usercliente, de que pueda atender comunicaciones simultáneas en ellaboas es necesario utilizar hilos o threads.

Un usuario HTTP establecidohilos que atender varias peticiones simultáneamente. Para ello, tenemos que ser capaces de modificar su código para que pueda ofrecer varios hilos de ejecución. Pues se hace de la siguiente manera:

- El bloc principal (o was, el que inicia la aplicación) creará al menos un worker que gestionará la cola de peticiones que llegue a nuestra plataforma.
- Cuando se reciba una, la asignará un worker libre para atenderla lo respuesta. Pero en lugar de esperarla al hacerlo, el hilo principal creará un nuevo hilo, para que la despache, por el contrario, se pondrá a la espera de nuevas peticiones.

Esquemáticamente, el código del hilo principalmente el siguiente aspecto:

```
fn() {
    workerCola() = new WorkerPool(poolTo);
    while (true) {
        // espera una peticiones, si no asigna un worker libre para su respuesta.
        // Cuando se asigna un worker, se crea un nuevo hilo para atenderla por un workerCliente que se asigne
        // cola si nuevo hilo para despachada por un workerCliente que se asigne
        this = new WorkerExpress(workerCliente);
        this.start();
    }
} catch (IOException ex) {
}
```

Hemos de clase **WorkerExpress** sobre una extensión de la clase Thread.Se, vuce, vuyo constructor almacenará el workerMantes que nombre un vino variable local utilizada luego por su método run() para hacer la respuesta:

```
class WorkExpress(worker workerCli, Thread ) {
    private Socket workerCliente;

    public WorkExpress(Socket[thread] workerCliente) {
        this.workerCliente = workerCliente;
    }

    public void run() {
        try {
            // Creamos su petición por el workerCliente
        } catch (IOException ex) {
        }
    }
}
```

Hasta aquí, tendríamos todo un Servidor HTTP capaz de gestionar peticiones concurrentes de manera más eficiente.

Aquí tienes un ejemplo mucho más avanzado de un servidor HTTP simplado que usa un pool de hilos para gestionar las concurrencias:



Hilos publicado con www.anonymousemail.me/anonymous-email.net/anonymous-email.net

Monitorización de tiempos de respuesta

Una cuestión muy importante para analizar el rendimiento y comportamiento de una aplicación distribuida es monitorizar los tiempos de respuesta del servidor. Los tiempos que intervienen desde que un cliente realiza una petición al servidor hasta que recibe su resultado son dos:

- Tiempo de procesamiento. Es el tiempo que transcurre transcurre para procesar la petición del cliente y enviar los datos.
- Tiempo de transmisión. Es el tiempo que tardan los mensajes en llegar y ser recibidos desde ida y vuelta.

¿Cómo podemos medir estos tiempos?

Para medir el tiempo de procesamiento hacer inciso el tiempo que transcurre en el servidor para procesar la solicitud del cliente. Por ejemplo, un pequeño código permite medir este tiempo en milisegundos:

```
// anota la hora/s del reloj del sistema antes de iniciar el procesamiento
long tiempoInicio = System.currentTimeMillis();
// procesamiento de la petición del cliente...

// anota la hora/s del reloj del sistema al finalizar el procesamiento
long tiempoFin = System.currentTimeMillis();
long tiempoProceso = tiempoFin - tiempoInicio;
System.out.println("Tiempo en procesar petición: " + tiempoProceso);
```

Para medir el tiempo de transmisión será necesario que el servidor envíe un mensaje con el tiempo del sistema al cliente. El cliente al recibir el mensaje debe restarse su tiempo de sistema y compararlo con el tiempo recibido en el mensaje.

Para poder comparar los tiempos de respuesta de dos equipos se imprescindible que ese tiempo de sistema esté sincronizado a través de cualquier servicio de tiempo (NTP). En equipos Windows por ejemplo, la sincronización de los tiempos se realiza automáticamente.

Aquí tienes un artículo muy completo sobre [cómo sincronizar tu reloj con un servidor NTP](#).



Obra publicada con [Leanpub](#) bajo [Licencia Creative Commons Reconocimiento-NoComercial-CompartirIgual 4.0 Internacional](#)

Ejemplo de monitorización del tiempo de transmisión.

Vamos a ver un ejemplo de cómo calcular el tiempo de transmisión de un recurso desde un servidor web. Para ello, vamos a modificar el cliente HTTP realizado anteriormente con las clases URL y URLConnection, para añadir la función URLConnection.getDate() que proporciona el instante en el que el servidor inicia la transmisión de su respuesta al cliente. Esa marca es enviada por el servidor al cliente mediante la cabecera Date, en igual que la cabecera que ahora analizaremos sobre el tipo de contenido, y que acabos aceptar con el método URLConnection.getContentType().

Desde el siguiente enlace puedes descargar el proyecto casa completo.



Alojado en GitHub - https://github.com/josemariadiaz/ejemplo-tiempo-transmision

Untitled

Made in an educational tool by Microsoft Education. Please report inappropriate content. Terms of use